

The Python **random** Module

A practical guide with commented, runnable examples

Python's built-in **random** module generates pseudo-random numbers and performs random operations on sequences (lists, tuples, strings). It is part of the standard library, so no installation is required — just `import random`.

It is called **pseudo**-random because the numbers are produced by a deterministic algorithm (the **Mersenne Twister**) seeded from the system clock or OS entropy by default. Given the same seed, the sequence of "random" numbers is always the same — which is very useful for reproducible tests and debugging.

Where it's used

- Shuffling data for training/test splits in machine learning
 - Simulations, games, and Monte Carlo methods
 - Picking random samples for testing or A/B experiments
 - Generating test data or passwords (non-cryptographic use only)
- For anything security-sensitive (tokens, passwords, cryptographic keys), use the `secrets` module instead — `random` is predictable if an attacker learns the seed.

1. Basic Float Generators

random.random()

Returns a random float N such that $0.0 \leq N < 1.0$.

```
import random

# random.random() returns a float in the half-open range [0.0, 1.0)
value = random.random()
print(value) # e.g. 0.7942481955504642
```

random.uniform(a, b)

Returns a random float N such that $a \leq N \leq b$ (inclusive of both ends, subject to floating-point rounding).

```
import random

# random.uniform(a, b) -> float between a and b, inclusive
temperature = random.uniform(20.0, 30.0)
print(f"Simulated temperature: {temperature:.2f}C")
```

random.gauss(mu, sigma) / random.normalvariate(mu, sigma)

Returns a float sampled from a Gaussian (normal) distribution with mean mu and standard deviation sigma. Useful for simulating natural, bell-curve-shaped data.

```
import random

# Simulate exam scores centered around 75 with a spread of 10
score = random.gauss(mu=75, sigma=10)
print(round(score, 1))
```

2. Random Integers

random.randint(a, b)

Returns a random integer N such that $a \leq N \leq b$. Both ends are inclusive.

```
import random

# randint(a, b) is INCLUSIVE on both ends
dice_roll = random.randint(1, 6)
print(dice_roll) # any of 1, 2, 3, 4, 5, 6
```

random.randrange(start, stop, step)

Works like Python's `range()`: stop is EXCLUDED, and you can pass a step to only pick from certain values.

```
import random

# randrange(stop) -> integer from 0 up to (but not including) stop
print(random.randrange(10)) # 0-9

# randrange(start, stop) -> integer from start up to (not including) stop
print(random.randrange(5, 10)) # 5-9
```

```
# randrange(start, stop, step) -> only even numbers between 0 and 20  
print(random.randrange(0, 20, 2)) # 0, 2, 4, ..., 18
```

■ **Common mistake:** `randint(a, b)` is inclusive of `b`, but `randrange(a, b)` is NOT. Mixing these up is a classic off-by-one bug.

3. Working with Sequences

random.choice(seq)

Returns a single random element from a non-empty sequence.

```
import random

colors_list = ["red", "green", "blue", "yellow"]

# choice() picks exactly ONE item from the sequence
winner = random.choice(colors_list)
print(winner)
```

random.choices(population, weights=None, k=1)

Returns a list of k elements chosen **with replacement** (the same item can be picked more than once). Supports optional weights to bias the selection.

```
import random

items = ["common", "rare", "legendary"]
# Weighted loot drop: "common" is far more likely than "legendary"
weights = [80, 15, 5]

# k=5 draws, WITH replacement -> duplicates are possible
loot = random.choices(items, weights=weights, k=5)
print(loot) # e.g. ['common', 'common', 'rare', 'common', 'common']
```

random.sample(population, k)

Returns a list of k **unique** elements chosen **without replacement**. Use this instead of choices() when you don't want duplicates.

```
import random

deck = list(range(1, 53)) # simulate a 52-card deck

# sample() draws WITHOUT replacement -> no duplicates in the result
hand = random.sample(deck, k=5)
print(hand) # 5 distinct "cards"
```

random.shuffle(seq)

Shuffles a mutable sequence **in place** (returns None). Only works on mutable sequences like lists — not strings or tuples.

```
import random

playlist = ["song_a", "song_b", "song_c", "song_d"]

# shuffle() mutates the list directly; it does NOT return a new list
random.shuffle(playlist)
print(playlist) # order is now randomized
```

Quick comparison

Function	Replacement?	Returns	Typical use
----------	--------------	---------	-------------

choice(seq)	n/a (1 pick)	single item	Pick one random item
choices(seq, k=n)	With replacement	list (len n)	Weighted / repeated draws
sample(seq, k=n)	Without replacement	list (len n)	Unique random subset
shuffle(seq)	n/a	None (in place)	Randomize full order

4. Reproducibility with seed()

By default, the random module seeds itself using system entropy, so every run produces a different sequence. Calling `random.seed(value)` resets the internal generator so the SAME sequence of "random" values is produced every time — essential for reproducible tests, tutorials, and debugging.

```
import random

# Fixing the seed makes the output deterministic and repeatable
random.seed(42)
print(random.randint(1, 100)) # always the same value when seed=42
print(random.randint(1, 100)) # always the same second value too

# Without a fixed seed, results differ on every program run
random.seed() # re-seeds from system entropy (current time / OS source)
print(random.randint(1, 100)) # unpredictable again
```

■ Fix the seed at the very start of a script (e.g. `random.seed(0)`) whenever you need identical output across runs — common in ML experiments and unit tests.

5. Worked Example: Simple Card Game

A small, fully commented script that ties several functions together.

```
import random

# Fix the seed so this demo prints the same result every time it's run
random.seed(7)

def build_deck():
    """Build a standard 52-card deck as a list of (rank, suit) tuples."""
    ranks = ["2", "3", "4", "5", "6", "7", "8", "9", "10",
             "J", "Q", "K", "A"]
    suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
    return [(rank, suit) for suit in suits for rank in ranks]

def deal_hands(deck, num_players=2, cards_per_hand=5):
    """Shuffle the deck, then deal hands without any card repeating."""
    random.shuffle(deck) # randomize order IN PLACE
    hands = []
    for _ in range(num_players):
        hand = [deck.pop() for _ in range(cards_per_hand)] # remove dealt cards
        hands.append(hand)
    return hands

deck = build_deck()
player_hands = deal_hands(deck, num_players=2, cards_per_hand=5)

for i, hand in enumerate(player_hands, start=1):
    print(f"Player {i}: {hand}")

# Pick a random player to go first
first_player = random.choice(range(1, len(player_hands) + 1))
print(f"Player {first_player} goes first!")
```

6. Function Reference

Function	Signature	Description
<code>random()</code>	<code>random.random()</code>	Float in [0.0, 1.0)
<code>uniform()</code>	<code>random.uniform(a, b)</code>	Float between a and b, inclusive
<code>randint()</code>	<code>random.randint(a, b)</code>	Integer in [a, b], inclusive
<code>randrange()</code>	<code>random.randrange(start, stop, step)</code>	Integer like range(), stop excluded
<code>choice()</code>	<code>random.choice(seq)</code>	One random element from seq
<code>choices()</code>	<code>random.choices(pop, weights, k)</code>	k elements, with replacement
<code>sample()</code>	<code>random.sample(pop, k)</code>	k unique elements, no replacement
<code>shuffle()</code>	<code>random.shuffle(seq)</code>	Shuffle list in place, returns None
<code>seed()</code>	<code>random.seed(a)</code>	Initialize generator for reproducibility
<code>gauss()</code>	<code>random.gauss(mu, sigma)</code>	Float from a normal distribution

Guide generated for interview/study reference. All code samples are self-contained and runnable as-is with Python 3.